



Government
of Canada

Gouvernement
du Canada

ESDC Development Community

Managing Merge Requests, Using Code Review

Le texte français est donné à la suite.

Overall best practices

- Be kind.
- Accept that many programming decisions are opinions. Discuss tradeoffs, which you prefer, and reach a resolution quickly.
- Ask questions; don't make demands. ("What do you think about naming this :user_id?")
- Ask for clarification. ("I didn't understand. Can you clarify?")
- Avoid selective ownership of code. ("mine", "not mine", "yours")
- Avoid using terms that could be seen as referring to personal traits. ("dumb", "stupid"). Assume everyone is attractive, intelligent, and well-meaning.
- Be explicit. Remember people don't always understand your intentions online.
- Be humble. ("I'm not sure - let's look it up.")
- Don't use hyperbole. ("always", "never", "endlessly", "nothing")
- Be careful about the use of sarcasm. Everything we do is public; what seems like good-natured ribbing to you and a long-time colleague

might come off as mean and unwelcoming to a person new to the project.

- Consider one-on-one chats or video calls if there are too many “I didn’t understand” or “Alternative solution:” comments. Post a follow-up comment summarizing one-on-one discussion.
- If you ask a question to a specific person, always start the comment by mentioning them; this will ensure they see it if their notification level is set to “mentioned” and other people will understand they don’t have to respond.

Responsibility of the *author*

- Keep your changes small with a clear scope.
- Describe the changes in the Pull Request. Link it to a task.
- New functions and conditions come with new unit tests. Changed functions and conditions don't affect existing unit tests.
- Complex sections of code contains comments.

Having your code reviewed

Please keep in mind that code review is a process that can take multiple iterations, and reviewers may spot things later that they may not have seen the first time.

- The first reviewer of your code is you. Before you perform that first push of your shiny new branch, read through the entire diff. Does it make sense? Did you include something unrelated to the overall purpose of the changes? Did you forget to remove any debugging code?

- Be grateful for the reviewer's suggestions. ("Good call. I'll make that change.")
- Don't take it personally. The review is of the code, not of you.
- Explain why the code exists. ("It's like that because of these reasons. Would it be more clear if I rename this class/file/method/variable?")
- Extract unrelated changes and refactorings into future merge requests/issues.
- Seek to understand the reviewer's perspective.
- Try to respond to every comment.
- The merge request author resolves only the threads they have fully addressed. If there's an open reply, an open thread, a suggestion, a question, or anything else, the thread should be left to be resolved by the reviewer.
- Push commits based on earlier rounds of feedback as isolated commits to the branch. Do not squash until the branch is ready to merge. - Reviewers should be able to read individual updates based on their earlier feedback.
- Assign the merge request back to the reviewer once you are ready for another round of review. If you do not have the ability to assign merge requests, @ mention the reviewer instead.

Responsibility of the *reviewer*

- When approving a merge, you are just as responsible for the changes as the person who made them. So you should understand them just as well.
- Only review changed lines.

- Anyone on the project can raise concerns (blocking) on a PR/MR, even if they are not assigned to it.
- The goal of a PR/MR should ultimately be knowledge transfer.

Reviewing the code

Understand why the change is necessary (fixes a bug, improves the user experience, refactors the existing code). Then:

- Try to be thorough in your reviews to reduce the number of iterations.
- Communicate which ideas you feel strongly about and those you don't.
- Identify ways to simplify the code while still solving the problem.
- Offer alternative implementations, but assume the author already considered them. ("What do you think about using a custom validator here?")
- Seek to understand the author's perspective.
- If you don't understand a piece of code, say so. There's a good chance someone else would be confused by it as well.
- Do prefix your comment with "Not blocking:" if you have a small, non-mandatory improvement you wish to suggest. This lets the author know that they can optionally resolve this issue in this merge request or follow-up at a later stage.
- After a round of line notes, it can be helpful to post a summary note such as "LGTM :thumbsup:", or "Just a couple things to address."
- Assign the merge request to the author if changes are required following your review.
- Consider using the Squash and merge feature when the merge request has a lot of commits. When merging code a maintainer should only use the squash feature if the author has already set this option or if the

merge request clearly contains a messy commit history that is intended to be squashed.

Finding balance

One of the most difficult things during code review is finding the right balance in how deep the reviewer can interfere with the code created by a reviewee.

- Finding bugs and improving code style is important, but thinking about good design is important as well. Building abstractions and good design is what makes it possible to hide complexity and makes future changes easier.
- Asking the reviewee to change the design sometimes means the complete rewrite of the contributed code. It's usually a good idea to ask another maintainer or reviewer before doing it, but have the courage to do it when you believe it is important.
- There is a difference in doing things right and doing things right now. Ideally, we should do the former, but in the real world we need the latter as well. A good example is a security fix which should be released as soon as possible. Asking the reviewee to do the major refactoring in the merge request that is an urgent fix should be avoided.
- Doing things well today is usually better than doing something perfectly tomorrow. Shipping a kludge today is usually worse than doing something well tomorrow. When you are not able to find the right balance, ask other people about their opinion.

FAQ

How do smaller changes help me and the project?

Smaller changes allow each change to be individually assessed and validating ensuring that specific set of code works exactly as the developer intended it to. It also helps a reviewer of the code, the smaller the change, the faster the reviewer can approve it meaning the faster that change gets added to the shared source allowing all the developers to work with that change. Later when going through history trying to identify issues, the smaller changes will help identify the direct intention of the code change allowing a developer to identify how the change may relate to the issue they are dealing with.

Why do code reviews matter?

The primary goal of a code review should be collaboration. The collaboration allows the *reviewer* and *author* of the code change to learn more about the project, enabling them both to make better changes to the project in the moment and in the future.

How do I keep reviews from taking up all my time?

- **Keep the changes small!** If the change is too large and could have been broken apart, reject it.
- **Don't validate the code** in your review. Let the pipeline do that work for you. Focus on learning what it is the *author* is trying to do, and check that what they did makes sense.
- If you don't understand the change, **ask for more details**. Don't spend time trying to figure it out, it should be simple enough (or explained in comments) to understand in a minute or two.

Attribution

Some content copied from [GitLab Code Review Guidelines](#)

Texte français:

Bonnes pratiques générales

- Soyez bienveillant.
- Acceptez que de nombreuses décisions de programmation relèvent d'opinions. Discutez des compromis et trouvez rapidement une solution.
- Posez des questions au lieu d'imposer des exigences. (« Que penses-tu de nommer ceci :user_id ? »)
- Demandez des éclaircissements. (« Je n'ai pas compris. Peux-tu préciser ? »)
- Évitez le sentiment d'appropriation exclusive du code. (« mon code », « ton code »)
- Évitez les termes pouvant être perçus comme visant des traits personnels (« idiot », « stupide »). Partez du principe que tout le monde est compétent et bien intentionné.
- Soyez explicite. N'oubliez pas que vos intentions ne sont pas toujours évidentes par écrit.
- Restez humble. (« Je n'en suis pas certain - vérifions ensemble. »)
- Évitez les hyperboles (« toujours », « jamais », « rien »).
- Soyez prudent avec le sarcasme. Nos échanges sont publics; une plaisanterie amicale envers un collègue de longue date peut paraître blessante pour un nouveau venu.

- Envisagez un échange direct ou un appel vidéo s'il y a trop de commentaires d'incompréhension. Publiez ensuite un commentaire résumant la discussion.
- Si vous posez une question à une personne précise, mentionnez-la au début du commentaire afin qu'elle reçoive la notification et que les autres sachent qu'ils n'ont pas à répondre.

Responsabilités de l'auteur

- Gardez vos changements de taille réduite avec une portée bien définie.
- Décrivez les modifications dans la Pull Request / Merge Request. Liez-la à une tâche.
- Les nouvelles fonctions et conditions doivent s'accompagner de nouveaux tests unitaires. Les fonctions modifiées ne doivent pas briser les tests existants.
- Les sections de code complexes doivent être commentées.

Faire réviser son code

Gardez à l'esprit que la révision de code est un processus itératif et que les réviseurs peuvent relever des éléments plus tard qu'ils n'avaient pas vus au premier abord.

- Vous êtes le premier réviseur de votre code. Avant de pousser votre nouvelle branche, examinez l'ensemble du diff. Est-ce cohérent ? Y a-t-il du code superflu ou du code de débogage oublié ?
- Accueillez favorablement les suggestions des réviseurs. (« Bonne remarque, je fais la modification. »)
- Ne prenez pas les remarques personnellement. La révision porte sur le code, pas sur votre personne.

- Expliquez la raison d'être du code. (« C'est ainsi pour telle raison. Serait-ce plus clair si je renomme cette classe/méthode/variable ? »)
- Isolez les changements non reliés et les refactorisations dans de futures demandes de fusion ou tickets.
- Cherchez à comprendre la perspective du réviseur.
- Efforcez-vous de répondre à chaque commentaire.
- L'auteur ne résout que les fils de discussion qu'il a entièrement traités. En cas de question ou suggestion ouverte, le fil doit être laissé à la résolution du réviseur.
- Poussez les commits de rétroaction de manière isolée sans les écraser (squash) immédiatement, afin que les réviseurs puissent voir l'évolution.
- Réassignez la demande de fusion au réviseur lorsque vous êtes prêt pour un nouvel examen, ou mentionnez-le (@mention).

Responsabilités du *réviseur*

- En approuvant une fusion, vous partagez la responsabilité du code avec son auteur. Vous devez donc le comprendre tout aussi bien.
- Ne réviser que les lignes modifiées.
- Tout intervenant du projet peut soulever des points bloquants sur une PR/MR, même s'il n'y est pas formellement assigné.
- L'objectif ultime d'une PR/MR doit être le transfert de connaissances.

Réviser le code

Comprenez pourquoi le changement est nécessaire (correction de bogue, amélioration UX, refactorisation). Ensuite :

- Soyez rigoureux pour limiter le nombre d'itérations.

- Exprimez clairement les points qui vous tiennent à cœur et ceux qui sont secondaires.
- Cherchez à simplifier le code tout en répondant au besoin.
- Proposez des implémentations alternatives avec bienveillance.
- Tentez de comprendre le point de vue de l'auteur.
- Si un extrait de code vous paraît obscur, dites-le.
- Préfixez vos remarques non bloquantes par « Non bloquant : » pour indiquer qu'il s'agit d'une suggestion facultative.
- Après vos commentaires détaillés, publiez une note de synthèse comme « LGTM :thumbsup: » ou « Juste quelques points à revoir ».
- Réassignez la demande à l'auteur si des corrections sont requises.
- Utilisez l'option d'écrasement de commits (*Squash and merge*) si l'historique de la branche est chargé et désordonné.

Trouver le juste équilibre

L'un des aspects les plus délicats de la révision de code consiste à doser le niveau d'intervention sur le travail d'autrui.

- Détecter les bogues et soigner le style est crucial, mais concevoir une architecture propre l'est tout autant. Les bonnes abstractions facilitent les évolutions futures.
- Demander une refonte de conception peut signifier réécrire le code soumis. Discutez-en au préalable avec un autre pair, mais ayez le courage de le proposer si c'est pertinent.
- Faire les choses parfaitement et répondre à l'urgence sont deux impératifs distincts. Pour un correctif de sécurité urgent, évitez d'exiger une refactorisation majeure.
- Une solution satisfaisante livrée aujourd'hui vaut souvent mieux qu'une solution parfaite livrée trop tard. En cas de doute, sollicitez

l'avis de vos pairs.

FAQ

En quoi les petits changements aident-ils mon projet ?

Les petites modifications permettent d'évaluer et de valider chaque changement de manière isolée. Elles accélèrent considérablement le travail de révision et l'intégration continue. Lors de l'analyse rétrospective de l'historique Git, les commits ciblés facilitent grandement la compréhension des intentions passées.

Pourquoi la révision de code est-elle essentielle ?

Le but premier de la révision de code est la collaboration et le partage de connaissances au sein de l'équipe, assurant ainsi une qualité logicielle durable.

Comment éviter que les révisions ne monopolisent tout mon temps ?

- **Gardez les changements petits !** Si une PR est trop volumineuse et divisible, demandez à la scinder.
- **Ne validez pas l'exécution manuellement.** Laissez le pipeline de tests automatisés faire son travail. Concentrez-vous sur la logique et la cohérence métier.
- Si le code n'est pas clair, **demandez des explications.** Ne perdez pas de temps à deviner; le code ou les commentaires doivent être compréhensibles en quelques minutes.

Attribution

Certains éléments sont adaptés des [GitLab Code Review Guidelines](#).

